tramdag documentation

Contents

- Semantics
- Propensity-centered VC: `center=True` (R-learner orthogonalization)
- Validation

Upstream candidates for zuko

1. Analytic `call_and_ladj` for `BernsteinTransform`
2. Learnable/linear tails for `MonotonicRQSTransform`
3. Public inverse of `_constrain_theta`
4. Docstring fix: the Bernstein θ-shape off-by-one
5. `Logistic` in `zuko.distributions`

Anti-candidates (look upstreamable, are not)

tramdag — Interpretable Neural Causal Models (TRAM-DAGs) in PyTorch

> **Status: beta (0.x), under active development.** The API may change between releases until 1.0, so pin a version for reproducibility. Note that this README documents the **unreleased 0.4** API: the term constructors (`SI`/`CI`/`VC`), `scores`, `varying_coef` and `intercept_contributions` are not in `0.3.0` on PyPI. Until 0.4 ships, install from git to follow the docs below.

TRAM-DAGs model each variable of a structural causal model with a (transformation-model) flow: one triangular normalizing flow from iid standard-logistic latents to the observed variables. The triangular adjacency structure is exactly your causal DAG. Fit it **once** on observational data and answer all three rungs of Pearl's causal hierarchy — observational (L1), interventional (L2, the do-operator), and counterfactual (L3, Pearl abduction) — while keeping **interpretable effects**: every linear-shift coefficient is a log-odds ratio, exactly as in classical proportional-odds models.

> Beate Sick & Oliver Dürr, *Interpretable Neural Causal Models with TRAM-DAGs*, CLeaR 2025 (arXiv:2503.16206). This repo is the reference implementation (PyTorch, built on zuko); all of the paper's experiments are replicated here with pinned tests.

5-minute showcase: the Colab badge above fits the paper's bimodal benchmark live and walks L1 → L2 → L3, every answer checked against analytic ground truth. Further notebooks are available at `notebooks/` like the didactic walkthrough of the model: `notebooks/intro_tram_dag.py`.

Install

```
pip install tramdag            # latest release (PyPI)
pip install "git+https://github.com/tensorchiefs/tramdag.git@main"   # dev version (track main)
uv sync                        # or: dev setup from a clone (tests, experiments)
```

Pin the dev install to a commit for reproducibility, e.g. `...tramdag.git@<sha>`.

30 seconds of API

```python
from tramdag import CausalFlowDAG, ContinuousNode, OrdinalNode, I, LS, CS

spec = {  # the spec IS the labelled DAG
    "X1": ContinuousNode(),
    "X2": ContinuousNode(I("X1")),
    "T": OrdinalNode(2, LS("X1") + CS("X2")),  # 2 levels, column coded 0..1
    "Y": OrdinalNode(4, I("X1") + CS("X2") + LS("T")),
}
# train_df / val_df: DataFrames with one column per node
flow = CausalFlowDAG(spec)  # validates acyclicity, builds the flow

# fit() is one minibatch Adam loop; validation, schedules and early stopping
# attach through optimizer= and the callback hooks — the common recipes ship
# in tramdag.callbacks (docs/fitting.md)
from tramdag.callbacks import Logger, RestoreBest

best = RestoreBest(val_df)  # keep the best-validation weights
flow.fit(
    train_df,
    epochs=4000,
    batch_size=512,
    after_epoch_callbacks=[Logger(val_df, every=100), best],
    after_fit_callbacks=[best.restore],
)

# all-`ls` model? fit it classically instead: deterministic float64 L-BFGS,
# exact MLE matching statsmodels/R (see docs/fitting.md)
flow.fit_classical(train_df)  # raises on cs/ci/vc specs

flow.log_prob(df)  # L1: joint log-likelihood per row
flow.sample(1000)  # L1: observational sampling
flow.sample(1000, do={"T": 1})  # L2: interventional (graph mutilation)
flow.pmf(df, node="Y", do={"T": 1})  # L2: analytic interventional PMF
flow.density(df, node="X2", grid=grid, do={"X1": 0.5})  # ... and density, continuous nodes

u = flow.abduct(df)  # L3 step 1: latents from observations
cf = flow.sample(do={"T": 1}, u=u)  # L3 steps 2+3: counterfactuals

flow.ls_coefficients()  # interpret: per-edge log-odds-ratios (LS terms)
# per-parent partial effects of an additive complex intercept (centered):
# flow.intercept_contributions(df, "Y") on a CI("A", "B", allow_interaction=False)

# heterogeneous treatment effects: a small, penalized effect head beta(x)*T
# (VC term) with a first-class read-out — see docs/varying-coefficients.md
# e.g. CS("X1", "X2") + VC("X1", t="T") ->
# flow.varying_coef(df, "Y")               # beta(x): deterministic, y-free

flow.scores(df, node="Y")  # per-observation scores dl_i/dtheta
flow.effect_modifier_scan(df, "Y", t="T")  # which VC modifiers? (CUSUM
# scan from a cheap all-ls fit) — docs/scores.md

flow.save("flow.pt")
flow = CausalFlowDAG.load("flow.pt")
```

The model in detail: spec → math → networks

Per node, the transformation is additive on the latent (log-odds) scale — one intercept term `I` plus any number of shifts (notation: `docs/notation.md`):

`u = h_ϑ(x) + Σ β·x_pa + Σ g(x_pa) + (β₀ + b_ϑ(x_mod))·x_t`

term	math	what gets built	interpretability
`I()` / bare `I` / omitted	`h_ϑ(x)` — constant ϑ	`SimpleIntercept`: one free parameter vector, no network	the baseline transform
`I("A")`	`h_ϑ(a)(x)` — ϑ bends with the parent	`ComplexIntercept`: MLP `[8, 8] → n_params`	the parent reshapes the whole distribution; no single coefficient
`I("A","B")` (default `allow_interaction=True`)	`h_ϑ(a,b)(x)`	**one joint** MLP over both parents — they interact in ϑ	maximal flexibility
`I("A","B", allow_interaction=False)`	`h_ϑ(a)+ϑ(b)(x)`	one MLP **per parent**, parameter vectors summed in coefficient space	per-parent partial effects via `flow.intercept_contributions`
`LS("A")`	`β·a`	`Linear(width, 1)`, no bias — **one parameter per feature column** (one for a continuous parent, `levels` for a one-hot ordinal)	`exp(β)` is an odds ratio
`CS("A")`	`g(a)`, additive	`ComplexShift`: MLP `[64, 128, 64] → 1`	plot `g`
`CS("A","B")`	`g(a,b)` — joint	one MLP over the concatenated features	interaction *in the shift*
`CS("A") + CS("B")`	$g_1(a) + g_2(b)$	two MLPs, scalars added	GAM-style, each effect plottable
`VC("A","B", t="T")`	$(\beta_0 +$ `b_ϑ(a,b))·x_t`	scalar β_0 + zero-initialised penalized MLP `[16] → 1`; the treatment value multiplies, it never enters the net	β_0 ≈ constant effect, `flow.varying_coef` reads $\beta(x)$

A node takes **at most one `I` term with parents** — a term list is therefore always purely additive on the latent scale, and interactions exist only *inside* a term. Lists and `+` sums are interchangeable: `[LS("A"), CS("B")]` ≡ `LS("A") + CS("B")`.

Two knobs on the terms:

- **`transform=` on `I`** picks the basis of `h_ϑ` for a continuous node — `"bernstein"` (default, 20 coefficients, tails extrapolate with the boundary slope), `"spline"` (monotone RQ spline, 23 params at `bins=8`, fixed tail slope) or `"affine"` (2 params: the latent is exactly logistic). Ordinal nodes have no basis: their intercept is the cutpoint vector, `P(x ≤ k) = σ(ϑ_k − shift)`.
- **`units=` on `I`/`CS`/`VC`** sizes the term's network, e.g. `units=[16]` for one hidden layer of 16 neurons. The defaults match the PyTorch reference this package grew out of (buehlpa/TramDag, `tram_models.py`), **not** the TRAM-DAG paper's own R nets — so a replication sets `units=` and `activation=` explicitly, as the configs in `experiments/paper/` do.

Feature widths: a continuous parent enters raw (1 column), an ordinal parent one-hot (`levels` columns). Abduction is exact for continuous nodes and truncated-logistic for ordinal ones, so `flow.sample(u=flow.abduct(df))` reproduces `df` exactly / level-exactly.

There are two ways to fit the model: a stochastic deep learning optimizer (`fit`) and a 2nd order optimization like in the classical statistical models (`fit_classical`). The latter is more efficient for all-`ls` models (each node-conditional is then a classical transformation model). For more details, see the `docs/fitting.md` file.

Validation

Two layers, deliberately separate.

The framework's own test suite (`tests/`) measures the library against three inline data-generating processes it carries itself, so it needs no research code to run. Its strongest claim is an equality, not a similarity: an all-`ls` outcome node *is* an ordered-logit model, so the flow's MLE must match `statsmodels` `OrderedModel` on the same design matrix — it does, and the varying-coefficient acceptance bars (effect recovery, propensity centering) are pinned the same way. What the tests guarantee, and how each ground truth is obtained, is documented in `tests/README.md`.

The paper replications (`experiments/`) are separate: one self-contained script per dataset, with its hyperparameters in a sibling YAML file and its expected results committed under each area's `ground_truth/`. A dedicated workflow runs them and compares.

experiment	paper	demonstrates
`triangle.py` (`linear-ls`, `linear-cs`, `atan-cs`, `sin-cs`)	§6.1, C.3	LS coefficient recovery (β = 2, −0.2, +0.3), CS curve ≡ $-f(x_2)$ for non-monotone f
`triangle_mixed.py` (`linear-ls`, `exp-cs`)	§6.2	mixed data L1/L2 + the C.4 odds-ratio check (OR ≈ 7.4)
`vaca.py`	§5.1–5.2	the bimodal L1 case a default CNF misses; L2 $p(x_3 \mid do(x_2))$ against analytic means
`carefl.py`	§5.3	L3 counterfactual curves vs **analytic** truth
`validate_ls.py`	—	flow ≡ `statsmodels` ≡ R `MASS::polr` to ~4 decimals, on a frozen synthetic cohort with a known true effect

Sign note: ordinal shifts are *subtracted* here but *added* in the paper, so fitted ordinal weights are the paper's with flipped sign (each `truth.json` records both conventions). A figure-by-figure account of what is reproduced, and what is not (the competing CNF/NSF baselines), is in `experiments/paper/PAPER_COVERAGE.md`.

Training speed — schedules, per-node freezing, L-BFGS and device benchmarks: `docs/training-speed.md`.

Paper replication — every hyperparameter of the eight `experiments/paper` variants with its source in the paper's R code, the deviations, and the numbers (paper / previous protocol / now): `docs/paper-replication.md`.

Testing policy

See the `tests/README.md` file for more details.

Layout

```
src/tramdag/            spec.py transforms.py conditioners.py flow.py
                        scores.py                <- the framework, and nothing else
tests/                  unit tests, identities, acceptance bars, three inline DGPs
experiments/            research code, one directory per area, each self-contained:
                          paper/       the replications + generators + frozen data
                          benchmarks/  training-speed and machine measurements
                          misc/        the classical-MLE validation
                        paper/misc: <name>.py + <name>.yaml, data/,
                        ground_truth/, tests/, results/ (benchmarks reads the
                        other areas' data and pins no ground truth)
notebooks/              four executed examples: didactic intro, Colab demo,
                        additive-vs-joint intercepts, varying coefficients
docs/                   code-map.md (every class/function + all knobs),
                        fitting.md, notation.md, training-speed.md,
                        paper-replication.md, varying-coefficients.md,
                        scores.md
```

Implementation conventions (latent-scale signs, raw/one-hot parent encoding, log-space ordinal likelihood, seeding) are documented in `CLAUDE.md` and pinned by tests.

Citation

If you use `tramdag`, please cite the method paper:

```bibtex
@inproceedings{sick2025tramdag,
  title     = {Interpretable Neural Causal Models with TRAM-DAGs},
  author    = {Sick, Beate and D{\"u}rr, Oliver},
  booktitle = {Proceedings of the 4th Conference on Causal Learning and Reasoning (CLeaR)},
  series    = {Proceedings of Machine Learning Research},
  volume    = {275},
  year      = {2025},
}
```

Code map — every class and function in `src/tramdag/`

One entry per name, with its role and its place in the pipeline. Names in parentheses are private machinery: useful to know, not part of the API. The last section lists every training hyperparameter and where it lives.

`spec.py` — declare the model

Every term constructor has a pythonic name and a short alias; the two are the same object, so `LS is linear_shift`.

Name	Role
`Term`	One additive term of a node's transformation: the frozen triple `(effect, parents, options)`. `+` on terms builds plain lists. Effect-specific settings live in `options` and read as attributes (`term.penalty`, `term.units`, ...).
`simple_intercept()` / `SI`	The parentless intercept — the paper's SI. Free transform parameters, the same for every row. Carries the basis choice (`transform=`, default `"bernstein"`); extra keyword arguments pass straight to the transform class.
`complex_intercept()` / `CI`	The parent-conditioned intercept — the paper's CI: the parents reshape the monotone transform. Needs at least one parent. Also carries `units=` and `allow_interaction=` (joint vs. additive multi-parent intercept).
`intercept()` / `I`	The fallback: dispatches on its arguments to `SI` (no parents) or `CI` (parents). The bare names `I` and `SI` in a term list both mean the simple intercept.
`linear_shift()` / `LS`	Linear shift `beta * x` — the interpretable log-odds coefficient. Exactly one parent.
`complex_shift()` / `CS`	Complex shift: an MLP `g(x)`, additive on the latent scale. Several parents form one joint network.
`varying_coefficient()` / `VC`	Varying-coefficient shift `(beta0 + b_theta(mods)) * x_t` — the penalized treatment-effect head (issue #28). `center=` adds propensity centering (issue #30).
`ContinuousNode`	Continuous variable: monotone 1-D transform plus shifts. `terms` is the first positional argument.
`OrdinalNode`	Ordinal variable with `levels` classes: ordered logit (cutpoints) plus shifts.
`node_terms()` / `node_parents()`	Canonical term list / ordered de-duplicated parent names of a node.
`validate_and_sort()`	Edge-ownership validation plus Kahn topological sort. The returned order makes the flow triangular.
`spec_to_dict()` / `spec_from_dict()`	Checkpoint (de)serialization. A term serializes as `{effect, parents, options}` and nothing else, since `options` is already canonical. No compatibility shims: `spec_from_dict` rejects a term without `options`, the node constructors normalize the formula, and `validate_and_sort` checks the DAG.
(`_normalize_terms`, `_as_term`, `_intercept_basis`, `_options`, `_OPTION_DEFAULTS`)	Formula flattening and per-entry validation (a `+` sum nested in a list is rejected), the one-parented-`I` rule plus basis hoisting in one pass, canonical option storage.
(`_check_term`, `_check_node`, `_check_vc_term`, `_kahn_sort`)	

Name	Role
	The stages behind `validate_and_sort`: per-term validation (effect, LS arity, unknown parents), per-node edge-ownership bookkeeping, the VC-specific checks, and the topological sort.

`transforms.py` — the monotone map h and the ordinal transform

Name	Role
`StandardLogistic`	The TRAM base distribution: `log_prob`, `sample` (generator-aware), `icdf`.
`BernsteinUT`	Bernstein-polynomial transform (default basis, `n_coeffs=20`). Linear tail extrapolation follows the boundary derivative. `marginal_init_theta()` gives the calibrated start used by `calibrate(marginal_init=True)`.
`SplineUT`	Monotone rational-quadratic spline (`bins=8`). Tails extrapolate with a *fixed* slope — the structural reason spline trails Bernstein on tail-heavy data.
`AffineUT`	Monotone affine transform: the node-conditional is a logistic GLM.
`make_univariate_transform()`	Basis registry: name → transform instance.
`ordinal_cutpoints()`	Unconstrained `(n, K-1)` → increasing cutpoints with ±inf ends. Port of the original parametrization.
`ordinal_log_prob()`	`log P(Y=y)`, computed in log-space. Load-bearing: the naive sigmoid difference saturates in float32 and freezes nodes at init. Do not simplify.
`ordinal_pmf()` / `ordinal_sample()` / `ordinal_abduct()`	Class probabilities / latent → level / truncated-logistic latent recovery (Pearl step 1) for ordinal nodes.
`ordinal_marginal_init_theta()`	Cutpoint start that matches the empirical class frequencies (`calibrate(marginal_init=True)`).
(`_ScaledUT`, `_bounds`, `_log1mexp`)	Quantile pre-scaling base class (the inverse is zuko's, with its closed-form tail); per-level cutpoint intervals; stable `log(1-exp(x))`.

`conditioners.py` — the networks behind the terms

Default architectures replicate the PyTorch reference this package grew out of (buehlpa/TramDag, `tram_models.py`), so a fitted model stays comparable to it. They are **not** the TRAM-DAG paper's nets: the paper's R code uses `c(2, 25, 25, 2)` with sigmoid for the triangle experiments and a 10-100 tanh net for its CAREFL/VACA comparisons, so each config in `experiments/paper/` states `units=` and `activation=` itself.

Name	Term	Role
`SimpleIntercept`	bare `I`	Free parameter vector; no parents.
`ComplexIntercept`	`I(...)`	8-8 ReLU MLP from parent features to the transform parameters.
`LinearShift`	`LS`	`Linear(n, 1, bias=False)`. `.weight` is the interpretable coefficient; no bias because the intercept slot owns the constant.
`ComplexShift`	`CS`	64-128-64 ReLU MLP to one shift value.

Name	Term	Role
`VaryingCoef`	`VC`	`beta0 + b_theta(mods)` with a zero-initialized output layer and the L2 hook `l2()`. `beta()` evaluates the effect, `recenter()` re-splits `beta0`/`b_theta` after training (function-preserving).
`(_mlp)`	—	The one MLP builder: a stack of the given `units` with the term's `activation` (relu by default), then a bias-free output layer.

`flow.py` — the model

Name	Role
`CausalFlowDAG`	The flow: one `_Node` per variable in topological order. Construction seeds the weights (`seed=` is the reproducibility knob).
`calibrate()`	Once, from the training rows: transform ranges (train 5%/95% quantiles onto the domain), network-input min-max, the calibrated start (`marginal_init=True`). Called by the first fit; a checkpoint carries the flag.
`init_marginals()`	The calibrated start as an explicit step, callable any time: resets every simple intercept to its column's marginal (Bernstein map / ordinal class log-odds; spline and affine have no calibrated start). Not once-guarded — on a trained flow it restarts those intercepts. Calibrates a fresh flow's ranges itself.
`fit()`	Joint maximum likelihood: one minibatch Adam loop over all parameters (exact per node, because the NLL decomposes), final weights kept. Hooks: `optimizer=` (any torch optimizer, for schedulers), `after_epoch_callbacks=` (one callable or a list, `cb(flow, epoch, opt)` after every epoch, any `True` stops) and `before_fit_callbacks=`/`after_fit_callbacks=` around the loop — validation, schedules, early stopping and logging attach here; the common recipes ship in `callbacks.py`. `vc_ehat=` carries the out-of-fold propensities of centered VC terms. A second call continues training.
`fit_classical()`	Float64 full-batch L-BFGS for all-`ls` specs: deterministic, exact MLE, matches `statsmodels`/R `polr`. Refuses flexible specs.
`sample()`	Observational, interventional (`do=`, graph mutilation) and counterfactual (`u=`) sampling.
`abduct()`	Pearl step 1: recover the latents. Continuous exactly, ordinal by truncated draw.
`pmf()`	Analytic class probabilities of an ordinal node, with `do=` overrides.
`density()`	Analytic conditional density of a continuous node on a grid, with `do=` overrides — the continuous counterpart of `pmf`.
`log_prob()` / `nll()`	Joint per-row log-likelihood / mean per-node NLL diagnostic.
`node_log_prob()`	The per-node decomposition everything trains and evaluates through.
`varying_coef()`	Closed-form read-out `beta(x)` of a fitted VC term. Deterministic, y-free.
`scores()` / `effect_modifier_scan()`	Analytic per-observation scores and the CUSUM modifier scan (delegate to `scores.py`).
`intercept_contributions()`	Post-hoc GAM-style decomposition of a complex intercept into mean-centered per-term parts.
`ls_coefficients()`	The per-node linear-shift weights — the interpretable coefficients.
`design_matrix()`	Parent encoding as a DataFrame (`drop_first=` gives the classical statsmodels/`polr` design).

Name	Role
`to_matrix()`	The labeled meta-adjacency matrix of term effects.
`save()` / `load()`	Checkpoints with history and provenance (version, time, device). `load` requires a complete checkpoint and fails loudly otherwise.
`(_Node, _VCGroup)`	Per-node module (intercept + shift `ModuleDict` + VC bookkeeping); construction is `_build_intercept`/`_build_shifts`, and `theta_shift()` computes `(theta, shift)` through `_theta`/`_vc_shift`/`vc_column`.
`(_node, _encode_parent, _features, _tensorize, _generator, _dtype, _np_dtype, _feat_width, _slice_ehat, _term_cells)`	Node lookup with one shared error; parent encoding (continuous raw, ordinal one-hot); `_tensorize(df, cols=None)` for any column subset; seeded-generator, dtype, feature-width and adjacency-cell plumbing.
`(_fit_epoch)`	One shuffled pass over the rows; the epoch-mean train NLL per node goes to `history["train"]`.
`(_vc_ehat_train, _binary_p1, _vc_ehat_live, _vc_ehat_columns, _recenter_vc)`	The VC machinery: the caller's out-of-fold propensities as frozen training tensors (DML); live full-fit propensities for inference (recomputed under `do`, never cached); post-fit re-centering.
`(_is_all_ls, _covered_by_classical)`	Guard for `fit_classical`: every term an `LS`, or a parentless `I()` basis carrier.

`scores.py` — effect-modifier detection (issue #29)

Name	Role
`node_scores()`	Analytic, exact per-observation scores `psi_i = d l_i / d theta` for every `LS` weight and VC `beta0`. No autograd.
`effect_modifier_scan()`	Zeileis-Hornik fluctuation scan: order the treatment scores by each candidate, `sup
`sup_bb_pvalue()`	`P(sup
`(_dl_ds, _ls_score_columns, CRIT_5PCT)`	Closed-form latent-scale derivative; the LS/one-hot score-column builder; the 5% critical value 1.3581.

`callbacks.py` — the shipped `fit` callbacks

Name	Role
`Logger`	Prints one line per `every` epochs: summed train NLL, plus validation NLL when `val_df` is given.
`RestoreBest`	Snapshots the weights of the best summed validation NLL per epoch; `restore` (registered in `after_fit_callbacks`) loads them back before the VC re-centering.
`PerNodePlateau`	Per-node lr decay and freezing on each node's own validation NLL; stops the fit once every node froze. The pre-0.4 `fit(schedule="plateau")` recipe, opt-in. `step(nll, opt)` for a hand-computed NLL.
`per_node_adam()`	Adam with one `node`-tagged parameter group per node — the optimizer `PerNodePlateau` needs.

What is *not* in the package

The SCM generators, the frozen datasets and the replication scripts are research code and live in `experiments/`, outside the installed package — see `experiments/README.md`. The framework's own tests do not depend on them: they measure against three inline DGPs in `tests/conftest.py`.

Where every training hyperparameter lives

Everything that shapes a fit is either a keyword you pass or a documented default you can read at the call site. Nothing numeric is buried.

Knob	Where	Default
learning rate, batch size	`fit()`	1e-2 / 512 (in-repo callers state them explicitly anyway)
schedules, early stopping, logging	`fit(optimizer=, after_epoch_callbacks=)`	`tramdag.callbacks` ships `Logger`, `RestoreBest`, `PerNodePlateau`; anything else is torch's `lr_scheduler` and a few lines of callback (fitting.md)
calibrated init	`calibrate(marginal_init=)`	True (pure init, MLE unchanged; `False` = zuko's zero start; called by the first fit)
VC stage-1 propensities	`fit(vc_ehat=)`	required for a centered VC term, out of fold, computed by the caller
VC penalty and centering	`VC(penalty=, center=)`	1.0 / False (centering needs `fit(vc_ehat=)`)
L-BFGS budget	`fit_classical(max_iter=, tol=, history_size=)`	400 / 1e-9 (torch `tolerance_change`; `tolerance_grad` is off) / 50 — one full-batch run, no chunks
training budget	`fit(epochs=)`	**required** — a fixed default is wrong in both directions (training-speed)
network widths	`units=` on `I`/`CS`/`VC`	(8, 8) / (64, 128, 64) — parity with the PyTorch reference's default classes; VC's (16,) has no counterpart there and comes from the recovery measurement
activation	`activation=` on `I`/`CS`/`VC`	`"relu"` (the reference default classes); `"sigmoid"` and `"tanh"` are the paper's
transform basis	`I(transform=, **kwargs)` (extra kwargs go to the transform class)	`"bernstein"`, `n_coeffs=20` unconstrained coefficients (zuko ties two more control points on, so order 21); spline `bins=8` = zuko's NSF default (the domain is fixed at [-5, 5], `transforms.BOUND`)
shuffling / weight init	`fit(seed=)` / `CausalFlowDAG(seed=)`	init happens at construction — the constructor seed is the reproducibility knob
weight init	`CausalFlowDAG(init=)`	`"torch"` (`nn.Linear` Kaiming-uniform); `"glorot"` = Keras `Dense` default, glorot-uniform weights and zero biases — the paper's reference; decisive under its full-batch protocol
network inputs	`CausalFlowDAG(net_input_scaling=)`	`None` (raw parents); `"minmax"` scales the continuous parents of every net (CI/CS/VC modifiers) to [0, 1] by the training min–max, the reference's `scale_df` — LS and the VC treatment stay raw

Fitting a TRAM-DAG: how training works

This page is the technical reference for `CausalFlowDAG`. It explains how the module builds the flow and how it computes the likelihood. It describes the two fitting paths: the stochastic optimizer (`fit`) and the classical optimizer (`fit_classical`). It also covers the hooks, the memory model, and future directions for optimizer choice.

How the flow is built: one module, one sub-model per node

A `CausalFlowDAG` is a single `torch.nn.Module`, but it is **not one monolithic network**. At construction (`CausalFlowDAG.__init__`), it topologically sorts the DAG. It then builds an `nn.ModuleDict` with **one `_Node` sub-module per variable**. The nodes share *no parameters*. Each node owns the pieces for its own conditional `p(x_i | pa(x_i))`:

- an **intercept** that produces the transform parameters `θ`. This is `SimpleIntercept`, a free parameter vector that is data-independent. If the node has `ci` parents, it is `ComplexIntercept` instead, a small MLP whose output `θ` depends on those parents.
- a **monotone 1-D transform** `h` (`BernsteinUT` / `SplineUT` / `AffineUT`). The transform itself carries **no learnable weights**, only the fitted range buffers `xmin`/`xmax`. The `θ` from the intercept sets its shape entirely.
- a `ModuleDict` of **shift modules**, one per shift *term* — which is one per parent edge except for a joint `CS("a","b")`, a single module keyed `"a+b"` that owns both edges: `LinearShift` (`LS`, a single weight), `ComplexShift` (`CS`, an MLP) or `VaryingCoef` (`VC`, `beta0` plus a penalized head).

So is this one network or several? It is **one module, several independent per-node sub-models** (each itself a small intercept + shifts assembly). One module bundles them, and one optimizer trains them, but their parameters are disjoint. The DAG structure lives entirely in *which parents each node reads*. There is no edge weight matrix or shared trunk.

For a batch, `_Node.theta_shift` assembles a node's `θ` (from the intercept) and total `shift` (sum of its shift modules over the parent features). `_encode_parent` encodes the parent features: continuous parents raw, ordinal parents one-hot.

How the likelihood is computed

A TRAM-DAG maps iid standard-logistic latents `U` to the observed `X` in causal order. Node `i` reads only its parents (earlier variables, as *data*). Therefore the Jacobian of `U → X` is **triangular**, and its log-determinant is the sum of the per-node 1-D terms. The joint log-likelihood therefore **decomposes per node**:

```
log p(x) = Σ_i log p(x_i | pa(x_i))
```

`CausalFlowDAG.node_log_prob` computes one term per node and `log_prob` sums them. For a node, given `θ, shift` from `theta_shift`:

- **continuous** — change of variables through the monotone transform:

```
u = h(x; θ) + shift
log p(x | pa) = log f_logistic(z) + log |dz/dx|
```

That is, the term is the standard-logistic density at the latent `u` (`StandardLogistic.log_prob`) plus the transform's log-derivative. `ut.forward` returns this log-derivative as `ladj`. This is the 1-D Jacobian term that makes the result a proper density, not only a score.

- **ordinal** — an ordered-logit / proportional-odds head, `P(x ≤ k) = σ(θ_k − shift)`. `ordinal_log_prob` evaluates it as the log of the cutpoint-interval probability. The computation runs in log-space via `logsigmoid`/`log1mexp`, because the naive sigmoid difference underflows to exactly-zero gradients in float32.

The training loss is the summed per-node **mean** NLL over the batch (`Σ_i mean_rows(−log p(x_i | pa))`). In contrast, `log_prob` returns the per-row joint, which scores whole observations.

Consequence used by both optimizers: because parents enter as data, the per-node gradients are independent. Therefore a joint fit of the summed loss is identical to a separate fit of each node. This independence licenses per-node learning rates and freezing (a callback, below) and the all-`ls` classical fit.

Path A — stochastic optimization (`fit`)

`CausalFlowDAG.fit` is the general-purpose trainer. Any `cs`/`ci` edge requires it. Mechanics:

- **One optimizer over all parameters** — `Adam(lr=learning_rate)` by default, or any `torch.optim.Optimizer` you pass as `optimizer=`. The per-node NLLs have independent gradients, so one optimizer is exactly per-node training; one parameter group per node (for per-node learning rates) is something you build yourself when you want it.
- **Minibatches**: a fresh `torch.randperm` shuffle each epoch (`seed=` seeds it). The loss is the summed per-node mean NLL on the batch, plus the `VC` penalty.
- **`calibrate(train_df, marginal_init=True)`**, called by the first `fit`: the transform ranges from the train 5%/95% quantiles (each Bernstein/spline domain), the network-input min-max under `net_input_scaling="minmax"`, and the calibrated start — Bernstein nodes at the linear map onto the latent 5%/95% quantiles, ordinal cutpoints at the empirical class log-odds, a pure init that leaves the MLE unchanged. Call it yourself to switch the start off. A checkpoint carries the flag, so a loaded model is never recalibrated. The start itself is also a public step: `flow.init_marginals(train_df)` resets every Bernstein/ordinal simple intercept to its column's marginal (spline and affine have no calibrated start), any time — e.g. to restart a trained or loaded flow (`calibrate` won't, it is once-only).
- **Callback hooks** — `after_epoch_callbacks=` takes one callable or a list, each called as `cb(flow, epoch, optimizer)` after every epoch, once the epoch's train NLLs are in `flow.history["train"]`; every callback runs each epoch, and the fit stops after an epoch in which any returned `True`. `before_fit_callbacks=` / `after_fit_callbacks=` (`cb(flow, optimizer)`) bracket the loop — the after-fit hooks run *before* the `VC` re-centering, so a callback that swaps weights hands them to the re-centering. This is where validation (`flow.nll(val_df)`), schedules, snapshots, logging and coefficient trajectories live. The common recipes ship in `tramdag.callbacks`: `Logger` (epoch lines), `RestoreBest` (best-validation weights), `PerNodePlateau` + `per_node_adam` (per-node decay and freezing).
- **`vc_ehat=`**: the out-of-fold propensities a centered `VC` term needs, `{node: {t: array}}` with one value per training row (see varying-coefficients.md).

The recipes, as callbacks

The shipped ones first — best-validation weights plus progress lines is two imports:

```python
from tramdag.callbacks import Logger, RestoreBest

best = RestoreBest(val_df)
flow.fit(
    train_df,
    epochs=4000,
    after_epoch_callbacks=[Logger(val_df, every=50), best],
    after_fit_callbacks=[best.restore],
)
```

Anything else is a few lines of your own. A learning-rate schedule is torch's, stepped from the hook; the snapshot half of this snippet is what `RestoreBest` does inside, written out:

```python
import copy

opt = torch.optim.Adam(flow.parameters(), lr=1e-2)

# a learning-rate schedule: torch's, on the validation NLL
plateau = torch.optim.lr_scheduler.ReduceLROnPlateau(opt, factor=0.3, patience=30)

# best-validation weights (early stopping) — RestoreBest, spelled out
best = {"nll": float("inf"), "state": None}

def on_epoch(flow, epoch, opt):
    nll = sum(flow.nll(val_df).values())
    plateau.step(nll)
    if nll < best["nll"]:
        best.update(nll=nll, state=copy.deepcopy(flow.state_dict()))
    return opt.param_groups[0]["lr"] < 1e-5      # stop once the rate bottomed out

flow.fit(train_df, epochs=4000, batch_size=512, optimizer=opt,
         after_epoch_callbacks=on_epoch)
flow.load_state_dict(best["state"])
```

(One difference to `RestoreBest`: a post-fit `load_state_dict` skips the VC re-centering, so on a spec with a `VC` term prefer `after_fit_callbacks`.)

The per-node variant — one parameter group per node, each decayed on its own validation NLL and frozen (rate 0) once flat — is `tramdag.callbacks.PerNodePlateau` over a `per_node_adam(flow, lr)` optimizer; it was `fit(schedule="plateau", freeze_patience=)` before 0.4, left `fit` because it is a training strategy, not part of the model, and is measured in `experiments/benchmarks/bench_training.py`.

Freezing vs. parallelism (why one helps and the other usually does not)

A tempting argument says: "freezing a node saves time, so node computation costs time, so parallel computation of nodes will also save time." The premise is correct, but the conclusion does not follow. The two act through different mechanisms:

- **Freezing removes work.** A frozen node (rate 0 in its parameter group, or dropped from `node_log_prob(nodes=)` in a hand-written loop) runs no update; once *all* nodes are flat the callback stops the fit and deletes whole epochs. Parallelism can never do that (it can shrink time-per-epoch, not the number of epochs).
- **Parallelism only overlaps work.** It runs the same computations concurrently, and this helps *only if the hardware sits idle* during the sequential node loop. The hardware usually is not idle. Parallelism already exists one level down: each node's batched tensor ops run across all rows, and the BLAS/GPU backend already uses all cores. While node A's matmul runs, the cores are busy, so node B "at the same time" only time-slices the same cores. The result is contention, often slightly slower.

So the speed gain from freezing does **not** imply spare capacity for parallel work across nodes. The exception is the regime where per-node ops *under-utilize* the hardware: small batches, tiny models, or many small nodes on a big GPU where each kernel leaves it mostly idle. In that regime, node overlap can help. But the right tool is **fusion** (batch same-shaped nodes into one larger tensor op, or CUDA streams), not a threaded Python loop (which the GIL fights). That is the "vectorize/fuse the per-node loop" direction in Optimizer choice below. It is a conditional win in that regime, distinct from the unconditional win of freezing.

Benchmarks, schedule trade-offs, and the recommended self-stopping recipe are in training-speed.md. The worked walkthrough is `notebooks/intro_tram_dag.py`.

Path B — classical optimization (`fit_classical`)

`CausalFlowDAG.fit_classical` is the dedicated optimizer for **all-`ls`** models, where every node-conditional is a classical transformation model (ordered-logit / Colr). It raises on any `cs`/`ci`/`vc` term.

- **Full-batch, float64, L-BFGS** (strong-Wolfe line search). There are no minibatches, no schedule, and no early stopping. Therefore the fit is **deterministic** (same init → bit-identical) and lands on the **exact MLE** (matches `statsmodels`/R to ~1e-3 on well-identified coefficients).
- **Solver budget** (`max_iter=400`, `tol=1e-9`, `history_size=50`): one L-BFGS run with torch's own stopping rule — it ends when the NLL or the parameters move by less than `tol`, or at `max_iter`. The report's `n_iter` is torch's count and `converged` says whether a tolerance, not the cap, ended the run. `history_size` is the L-BFGS memory.
- **float64 is a transient compute mode**: `self.double()` upcasts parameters *and* the transforms' range buffers in one call. The fit runs in double. A `finally: self.float()` restores float32. The stored model stays float32, and so do `save`/`load`. The data path (`_tensorize`, `sample`, `pmf`) reads the model dtype, so it follows along.
- **Convergence**: the flag is true when torch's `tolerance_change` ended the run before `max_iter` did, and it is *advisory*. A Bernstein intercept and weakly-identified directions (rare one-hot levels, a flat treatment-effect ridge) continue to drift along zero-curvature valleys after the likelihood is at the optimum. Correctness comes from a comparison with classical software (`python -m misc.validate_ls classical`), not from the flag.
- Read the fitted coefficients with `ls_coefficients()`.

Warm-start handoff: classical fit, then keep training

`fit_classical` leaves the model at the MLE in float32, ready for any normal operation — and continuing with `fit()` from there **stays put**, which is both a check that the classical solution really is the optimum and a way to use it as a fast, principled initialization:

```
flow.fit_classical(train_df)                       # exact MLE, seconds
before = flow.ls_coefficients()["y"]
flow.fit(train_df, epochs=300, learning_rate=1e-3)  # a gentle Adam phase ...
after = flow.ls_coefficients()["y"]                 # ... barely moves
```

A small drift means the classical fit was already at the optimum. The same handoff warm-starts a varying-coefficient term from the classical linear-shift coefficient: fit the all-`ls` version of the node classically and copy the treatment weight into `flow.nodes[y].shifts[t].beta0`.

Memory and disk during fitting

Neither fitting path writes to disk. Everything during `fit` / `fit_classical` lives in RAM:

- model parameters and (for `fit`) the optimizer state
- the `history` dict (per-epoch, per-node train NLL), an in-memory attribute and not a file; whatever your callback records is yours

The **only** disk I/O in the module is the explicit, user-called `save` (`torch.save` of spec, options, state_dict, history, meta) and its counterpart `load`. So a fit produces no temp files, no checkpoints, and no scratch directory. Persistence is opt-in via `flow.save(path)`. (The *experiment scripts* write the `results/` and `docs/perf/` artifacts in this repo, not the library's fitting code.)

Optimizer choice — current and future

Today: **Adam** for flexible models, **L-BFGS** (float64) for all-`ls`. Because the NLL decomposes per node, several extensions are cheap through `optimizer=`. These extensions are worth consideration as the package matures:

- **IRLS / Fisher scoring for the all-`ls` path.** The classical fitting algorithm for proportional-odds / GLM-type models is iteratively reweighted least squares (Newton with the expected information). For the `ls` case, it will likely reach the MLE in *fewer, more stable* steps than L-BFGS. Also, the Fisher information that it computes is exactly the ingredient for the planned **standard-error table** (currently flagged as next in the CHANGELOG). It is a strong candidate to back `fit_classical` in a future version.
- **Second-order / curvature-aware methods for flexible nodes.** K-FAC or a Gauss-Newton approximation can help the `ci` / `cs` MLPs. However, full-batch quasi-Newton is a poor fit for them (minibatch noise also regularizes, which is why `fit_classical` refuses them).
- **Modern first-order variants.** AdamW (decoupled weight decay), RAdam (warmup- free), or Lion/Sophia are drop-in alternatives to Adam. The benchmark harness (`experiments/benchmarks/bench_training.py`, in `experiments/benchmarks/`) exists to evaluate exactly such swaps on time-to-target.
- **Per-node optimizer selection.** The loss decomposes, so with one parameter group per node different nodes can in principle use different optimizers (for example, L-BFGS for the `ls` nodes and Adam for an MLP node in a mixed model). This is a direction for mixed-flexibility DAGs.
- **Warm-start handoffs.** `fit_classical → fit` already works (the classical MLE as a fast, principled initialization). The reverse (Adam to escape, L-BFGS to polish) is a natural pattern to formalize.

These are *directions*, not commitments. The autoresearch loop (`docs/research/MISSION_autoresearch.md`) is a good venue to test the speed-oriented ones empirically before any adoption.

Notation

This page defines the notation. Every guide, docstring and notebook in this repository follows it.

Symbols

Random variables are capital Latin letters, for example Y. Their realizations — observed or sampled values — are lowercase Latin letters, for example y. The cumulative distribution function of Y is F_Y and the density is f_Y. A hat marks an estimate: $\hat{F}$ is the fitted distribution, F the true and often unknown one.

Sets and vectors are bold, for example $\mathbf{X} = [X_1, \ldots, X_J]$.

A single optimized parameter is a lowercase Greek letter, for example ϑ or β. A vector of parameters is bold lowercase, for example $\boldsymbol{\vartheta} = (\vartheta_1, \ldots, \vartheta_M)^\top$. A tuple that collects several parameter groups — for example all weight matrices of a network — is bold capital, for example $\boldsymbol{\Theta}$.

A function that a parameter vector defines carries it as a subscript: $h_{\boldsymbol{\vartheta}}$ is the monotone transformation with coefficients $\boldsymbol{\vartheta}$.

The model's symbols

Symbol	Meaning	In code
X_j, Y	the variables of the DAG	node names in the spec
$\mathrm{pa}(X_j)$	the causal parents of X_j	`node_parents`
U_j	the standard-logistic latent of node j	`u = flow.abduct(df)`
$h_{\boldsymbol{\vartheta}}$	the monotone transformation of a node	the `I` term; `theta` in docstrings
ϑ_k	the ordinal cutpoints, elements of $\boldsymbol{\vartheta}$	`ordinal_cutpoints`
β	a linear-shift coefficient; e^{β} is an odds ratio	`LS`; `flow.ls_coefficients()`
g_j	a complex shift, an MLP of one term's parents	`CS`
β_0	the constant treatment effect of a `VC` term	`beta0`
$b_{\boldsymbol{\Theta}}$	the penalized effect-modification network	the `VC` net; `units=` sets its layers
λ	the L2 weight on $b_{\boldsymbol{\Theta}}$	`penalty=`
σ	the logistic function	`torch.sigmoid`

Every node's model is one additive formula on the latent scale:

$$u = h(x \mid \mathrm{pa}(x)) = h_{\boldsymbol{\vartheta}(\cdot)}(x) + \sum_j \beta_j x_j + \sum_k g_k(x_k) + (\beta_0 + b_{\boldsymbol{\Theta}}(x_{\mathrm{mod}}))\, x_t.$$

For a continuous node the shifts are added, as written. For an ordinal node the shift is subtracted inside the sigmoid: $P(Y \le k \mid \mathrm{pa}) = \sigma(\vartheta_k - \mathrm{shift})$.

Plain-text fallback

Docstrings and terminal output cannot render Greek subscripts. There, `theta` stands for $\boldsymbol{\vartheta}$, `h_theta` for $h_{\boldsymbol{\vartheta}}$, `b_theta` for $b_{\boldsymbol{\Theta}}$ and `beta0` for β_0.

Paper replication — protocol, hyperparameters and results, per experiment

The eight variants under `experiments/paper/` replicate the TRAM-DAG paper (Sick & Dürr, CLeaR 2025, arXiv:2503.16206) against its own R code (`tensorchiefs/tram-dag`). This page lists, for each experiment, the DGP, the model, every hyperparameter with its source, what deviates from the paper and why, and the numbers: what the paper reports, what the previous protocol of this repository measured, and what the current 1:1 protocol measures.

Measured 2026-08-26 on `refactor/lean-fit` (seeds: DGP 42, init 7, shuffle 0) with the exact reference protocol — global plateau rule and Keras init; the 2026-08-25 numbers of `feat/followups` (per-node plateau, torch init; CI run 32878435001) are kept in the "R 1:1, torch init" column where they differ. "Previous" is the protocol before that branch: a 90/10 split of one draw, batch 512, the fit restarted in chunks (`fit_in_chunks`), and for VACA/CAREFL a second "polish" fit at a lower rate — a repository recipe, not the paper's. Its numbers come from the ground-truth files of that revision (`73f9f39`; `{max}` bounds were 2.5× the measurement).

The paper states four training numbers — n = 40000, 500 epochs, Adam, Bernstein order 20 — and shows results as figures. Where it gives no number, the "paper" column names the figure and what it shows.

What is common to all eight

item	reference	here
latent	standard logistic, shifts added on the continuous scale, subtracted for ordinal `P(Y<=k) = sigmoid(theta_k - shift)`	same (tests pin both signs)
Bernstein basis	`len_theta` unconstrained coefficients, `to_theta` softplus-cumsum, domain → [0, 1] with tangent-linear extrapolation outside; the domain comes from the train **5 %/95 % quantiles in the triangle scripts** (`quantile(..., c(0.05, 0.95))`, the min/max lines commented out) and from **min/max** in the comparison scripts (`scale_df`)	zuko Bernstein, `n_coeffs` unconstrained (zuko ties two control points on, so the free-parameter count matches), domain = train 5 %/95 % quantiles → [−5, 5], linear extrapolation. Triangle: a match up to the reparametrization [0,1] vs [−5,5], the tail rule and order 21 vs 19. VACA/CAREFL: quantiles instead of min/max — **deviation D1**
init	triangle scripts: `LinearMasked` layers with Keras `random_normal` (N(0, 0.05²)) on weights and biases, the LS `beta` layer included; comparison scripts: `layer_dense` default, glorot-uniform weights and zero biases	`init: normal` (triangle) and `init: glorot` (VACA/CAREFL), `CausalFlowDAG(init=)`; torch's default init remains the framework default and was the decisive deviation under the full-batch protocol, see VACA
optimizer	Keras Adam, eps 1e-7	torch Adam, eps 1e-8 — measured: no effect (VACA identical to four digits)
seeds	triangle scripts unseeded (`SEED = -1`); comparison scripts `dgp(..., seed=42)` on R's RNG	not replayable in torch, so every seed is a repo choice (42 for the DGP is kept as a nod to the reference)
calibrated start	none	`flow.calibrate(train, marginal_init=False)` in `helpers.fit_paper` (framework default is True)
	Keras dense with bias	

item	reference	here
intercept output layer		bias-free — **D3** (the same function class; the bias adds a constant to all unconstrained coefficients)
plateau rule (VACA/ CAREFL)	`update_learning_rate`: one optimizer, reduce when the summed validation NLL has not improved for 50 epochs (strict `<`), factor 0.1, min 1e-7	torch `ReduceLROnPlateau(patience=49, threshold=0, threshold_mode="abs", factor=0.1, min_lr=1e-7)` on `sum(flow.nll(val))` — the same rule, verified against torch's source

D1 (VACA/CAREFL only) and D3 remain; both were measured (below). The per-node plateau approximation of 2026-08-25 (D4) is gone with the lean `fit`. Two further repo choices are CAREFL's: a fresh 2500-row draw with a separate validation draw where the R run used CAREFL's file with `val = train`, and raw units where the R run sd-standardized x3/x4.

Triangle, continuous (`triangle.py`) — paper Sec. 6.1, App. C.3

DGP (`summerof24/triangle_structured_continous.R`): x1 ~ 0.5 N(0.25, 0.1²) + 0.5 N(0.73, 0.05²); h(x2 | x1) = 5 x2 + 2 x1 = u2; h(x3 | x1, x2) = 0.63 x3 − 0.2 x1 − f(x2) = u3, u ~ logistic. f: `linear` −0.3 x, `atan` 0.75 atan(5 (x + 0.12)), `sin` 2 sin(3x) + x. True weights in the flow's convention: β12 = +2, β13 = −0.2, β23 = +0.3 (linear only).

Model: `x1: SI`, `x2: SI + LS(x1)`, `x3: SI + LS(x1) + {LS(x2) | CS(x2)}`, Bernstein. CS net = reference `hidden_features_CS = c(2, 25, 25, 2)`, sigmoid (the ReLU line in `create_param_net` is commented out).

hyperparameter	paper / R code	previous	now
train / validation	`dgp(40000)` / `dgp(40000)`, two draws	36000 / 4000, one draw split	40000 / 40000, two draws
epochs	500	500 (in chunks of 10, Adam restarted each chunk)	500, one continuous run
lr	0.001 (`optimizer_adam()` default)	0.001	**0.004**, the CI deviation
batch	32 (Keras `fit()` default)	512	**256**, the CI deviation
`len_theta` / `n_coeffs`	20	20	20
schedule / early stop / init	none / none, final weights / random_normal	none / none / torch	none / none / `init: normal`
coefficient read-out	after every epoch (Keras loop)	at chunk boundaries	`fit(after_epoch_callbacks=)`, every epoch

Results

variant	metric	paper	previous	paper protocol, `init: normal` (batch 32 / lr 0.001)	CI config (batch 256 / lr 0.004), the pinned ground truth
linear-ls	β12 / β13 / β23	Fig. 14 trajectories; App. C.3 text: 1.98 / −0.21 / 0.26	1.983 / −0.145 / 0.285	1.987 / −0.170 / 0.282	1.988 / −0.171 / 0.295
linear-cs		Fig. 17: fitted CS is a straight line	−0.151; 0.507		−0.178; 0.088

variant	metric	paper	previous	paper protocol, `init: normal` (batch 32 / lr 0.001)	CI config (batch 256 / lr 0.004), the pinned ground truth
	β13; max |ĝ − (−f)| on [−1, 1]			−0.173; 0.122 (glorot init: 0.110, torch: 0.116)	
atan-cs	β13; cs max err	Fig. 7 right / 15 / 16: CS on −f; text: β12 = 2.07, β13 = −0.203	−0.149; 0.229	−0.168; 0.059 (glorot: 0.080, torch: 0.085)	−0.173; 0.077
sin-cs	β13; cs max err	Fig. 18: CS follows the non-monotone f, saturating at the grid ends	−0.185; 2.29	−0.195; 1.10 (glorot: 0.997, torch: 1.016)	−0.202; 1.024
all	|E[x3 | do(x1 = −1)] flow − DGP|	Figs. 16/17: histograms overlap	—	0.09–0.14	0.08–0.12
all	val NLL x3	—	2.4603–2.4731	2.4607–2.4764	2.4606–2.4764

β13 at −0.17 instead of −0.2: x1 has sd 0.254, so SE(β13) ≈ 0.036 at n = 40000 — inside one SE. The `sin-cs` error of 1.0 is the reference architecture's capacity: the two-unit bottleneck saturates at both ends of the grid exactly as in the paper's Fig. 18.

Triangle, mixed (`triangle_mixed.py`) — paper Sec. 6.2, App. C.4, App. B

DGP (`triangle_structured_mixed.R`): x1, x2 as above; x3 ordinal with four levels, cutpoints θ = (−2, 0.42, 1.02) (from the R code — the paper does not state them), level = #{k : u3 > θ_k + 0.2 x1 + f(x2)}; f: `linear` −0.3 x, `exp` 0.5 exp(x). The paper adds the ordinal shift, the flow subtracts it, so the fitted weights are β13 = −0.2, β23 = +0.3.

Model: `x1: SI`, `x2: SI + LS(x1)`, `x3: OrdinalNode(4, LS(x1) + {LS(x2) | CS(x2)})`. CS net = reference `c(2, 2, 2, 2)`, sigmoid.

hyperparameter	paper / R code	previous	now
train / validation	`dgp(40000)` / `dgp(10000)`	36000 / 4000 split	40000 / 10000, two draws
epochs, lr, batch, schedule, init	500, 0.001, 32, none, random_normal	500 (chunks of 10), 0.001, 512, torch	500 continuous, none, `init: normal`; **lr 0.004 / batch 256**, the CI deviation
`n_coeffs` (x1, x2)	20	20	20
odds-ratio check (App. C.4)	odds(x2 ≤ −1) under do(x1 += 1), theory $e^2 \approx 7.39$	40000 rows, seed 99	same
counterfactual PMF (App. B)	Fig. 10 explains why point counterfactuals fail for the ordinal node	2000 rows × 200 draws	same

Results

variant	metric	paper	previous	paper protocol, `init: normal`	CI config (batch 256 / lr 0.004), pinned
linear-ls	β12 / β13 / β23	Fig. 19: 2 / −0.2 / 0.3	1.983 / −0.268 / 0.322	1.987 / −0.246 / 0.319	1.988 / −0.243 / 0.306
linear-ls	odds ratio predicted / DGP	$e^2 \approx 7.39$; C.4 text: 7.74, CI [7.16, 8.38]	7.26 / 7.19	7.29 / 7.19	7.30 / 7.19
linear-ls	CF PMF TV vs analytic; P(true level) flow / analytic / mode bound	— (App. B qualitative)	0.084; 0.714 / 0.728 / 0.806	0.044; 0.718 / 0.728 / 0.806	0.038
exp-cs	β13; cs max err	Fig. 20: distributions match	−0.227; 0.885	−0.207; 0.142 (glorot: 0.071, torch: 0.126)	−0.202; 0.107
exp-cs	CF PMF TV; P(true level)	—	0.075; 0.924 / 0.921	0.023; 0.917 / 0.921	0.020

VACA / CNF benchmark (`vaca.py`) — paper Sec. 5.1–5.2, App. C.1

DGP (Sanchez-Martin et al. 2022, App. E.1): x1 ~ 0.5 N(−2, 1.5) + 0.5 N(1.5, 1); x2 = −x1 + N(0, 1); x3 = x1 + 0.25 x2 + N(0, 1). Gaussian noise — outside the logistic-latent family, so the all-`CI` flow has to fit it. Analytic target: E[x3 | do(x2 = a)] = −0.25 + 0.25 a. The paper's Sec. 5.2 text says $a \in \{-3, -2, 0\}$; its Fig. 5 panels and the R code (`vaca_triangle.r`) intervene at $a \in \{-3, -1, 0\}$ — the code is followed (the frozen `data/vaca/truth.json` holds the same three).

Model: `x1: SI`, `x2: CI(x1)`, `x3: CI(x1, x2)`, Bernstein `n_coeffs = 31` (reference M = 30, `len_theta = 31`), nets `dense(10, tanh) → dense(100, tanh) → dense(31)` (`comparison/utils.R::make_model`).

hyperparameter	R code	previous	now
train / validation	nTrain 2500 / `dgp(5000)`	18000 / 2000 split	2500 / 5000, two draws
epochs	10000 (`Figure_Triangle_Linear_Bimodal.R`, the sourcing script — not in our copy of the R code, so EPOCHS/M/nTrain for VACA rest on that reading)	400 in chunks of 50, then 120 "polish"	10000, one run
lr	0.001	0.01, polish 0.001	0.001
batch	full batch (one `apply_gradients` per epoch)	512	2500 = n_train
schedule	`update_learning_rate`: ReduceLROnPlateau on the summed val NLL, factor 0.1, patience 50, min_lr 1e-7, strict `<`	none	the same rule, global (torch's scheduler through `fit(optimizer=, after_epoch_callbacks=)`)
input scaling	`scale_df`: everything min-max to [0, 1]	raw	`net_input_scaling: minmax` (network inputs; targets D1)
n_compare	—	50000	50000

Results — the check is the flow's error against the analytic mean, not a pinned flow value.

metric	paper	previous	R 1:1, torch init, per-node plateau (08-25)	now: R 1:1, glorot, global plateau
|E[x3 | do(x2 = −3)] − (−1.0)|	Fig. 5: densities overlap	0.037	0.026	0.097
|E[x3 | do(x2 = −1)] − (−0.5)|	Fig. 5	0.012	— (do(−2) was scored: 0.034)	0.088
|E[x3 | do(x2 = 0)] − (−0.25)|	Fig. 5	0.010	0.077	0.019
sd(x1) flow vs analytic 2.0767	Fig. 4: bimodal x1 fitted (the default CNF fails)	2.074	error 0.0077	error 0.032
val NLL x3	—	1.4356	1.4351	1.4496

The 08-25 and 08-26 columns are not the same three interventions: the do(x2) set moved from the paper text's {−3, −2, 0} to the R code's {−3, −1, 0} (see the DGP paragraph above), so only do(x2 = −3) and do(x2 = 0) compare directly. At the earlier grid and with glorot the errors were 0.098 / 0.159 / 0.026 at this seed and 0.268 / 0.119 / 0.005 and 0.217 / 0.031 / 0.021 at init seeds 8 and 9 — the spread quoted below.

How the gap was traced, one change at a time under the exact protocol (inputs min-max scaled unless stated; do(x2) errors at the then-current grid −3 / −2 / 0):

variant	error	reading
raw parents into the nets	0.731 / 0.426 / 0.017	the tanh nets saturate: 40 % of the rows have |x1| > 2, 43 % |x2| > 2
per-node plateau, torch init	0.026 / 0.034 / 0.077	the 2026-08-25 approximation
global plateau (R's rule), torch init	0.523 / 0.334 / 0.129	the summed NLL keeps improving through x1 while x3 overfits
… without any plateau	0.562 / 0.354 / 0.142	the rule is what stops the full-batch overfit
… Adam eps 1e-7	0.523 / 0.334 / 0.129	no effect
… Bernstein on train min/max (D1 off)	0.154 / 0.012 / 0.062	helps, not the cause
… glorot init	0.035 / 0.006 / 0.007	the cause (a different random draw than the config's seed 7)
… glorot + min/max	0.035 / 0.056 / 0.057	

Under the exact protocol the result is **seed-sensitive at the off-manifold point** do(x2 = −3): 0.03–0.27 over four init draws, 0.005–0.026 at do(x2 = 0). The paper shows one run's densities. The committed bound is 2.5× the seed-7 measurement, and this table is the honest picture.

CAREFL benchmark (`carefl.py`) — paper Sec. 5.3, App. C.2

DGP (Khemakhem et al. 2021): x1, x2 ~ Laplace(0, 1/√2); x3 = x1 + 0.5 x2³ + ε; x4 = −x2 + 0.5 x1² + ε, ε ~ Laplace(0, 1/√2). Counterfactuals are analytic by noise abduction. Observation `x_obs`: noise (2, 1.5, 1.4, −1)

→ (2, 1.5, 5.0875, −0.5) in the SCM's units; the R code's `xObs.csv` is that point divided by CAREFL's sample sds (6.0104, 1.9114): (2, 1.5, 0.8465, −0.2616). The paper prints (2, 1.5, 0.81, −0.28), slightly off it (CAREFL's own text). **Before `feat/followups` the repository used the printed values as raw coordinates**, a 4σ-off point. The R run also scores in sd-standardized units, so its Fig. 6 y-axis is 6× ours for x3 — the `fig6_max_abs_err` entries are in raw units. Queries: x3^cf | do(x2 = α) and x4^cf | do(x1 = α), α ∈ [−3, 3].

Model: `x1, x2: SI`, `x3, x4: CI(x1, x2)`, Bernstein `n_coeffs = 31`, the same `make_model` nets as VACA.

hyperparameter	R code (`carefl_fig5.r`)	previous	now
train / validation	CAREFL's own `data/CAREFL_CF/X.csv` (`USE_EXTERNAL_DATA = TRUE`), x3/x4 sd-standardized, **`val = train`** — the plateau rule watched the training NLL	18000 / 2000 split	2500 rows drawn from the same SCM / a separate `dgp(5000)` draw, raw units — **repo choice**
epochs	7000	300 in chunks of 50 + 100 polish	7000
lr, batch, schedule, input scaling, init	0.001, full batch, plateau 0.1/50/1e-7, `scale_df`, glorot	0.01→0.001, 512, none, raw, torch	0.001, 2500, the same global rule, `minmax`, glorot
scoring	the single `x_obs`, curves over α (Fig. 6)	+ 300 held-out rows at α ∈ {−1.5, 0, 1.5}	same

Results

metric	paper	previous (wrong x_obs)	R 1:1, torch init, per-node plateau (08-25)	now: R 1:1, glorot, global plateau
Fig. 6 max |x3^cf error| over α at `x_obs`	Fig. 6: flow tracks the DGP curve	1.64	1.52 (at α = 3, x3 ≈ 17)	1.87 (at α = 3)
Fig. 6 max |x4^cf error|	Fig. 6	0.38	0.47	0.59
held-out CF MAE x3, α = −1.5 / 0 / 1.5	—	0.109 / 0.053 / 0.095	0.177 / 0.065 / 0.123	0.181 / 0.065 / 0.142 (seeds 8, 9: 0.158–0.171 / 0.069–0.084 / 0.136–0.154)
held-out CF MAE x4, α = −1.5 / 0 / 1.5	—	0.078 / 0.059 / 0.085	0.117 / 0.059 / 0.115	0.204 / 0.134 / 0.162 (0.172–0.178 / 0.113–0.129 / 0.145–0.160)
val NLL x3 / x4	—	1.376 / 1.345	1.395 / 1.373	1.403 / 1.419

The previous protocol trained on 7× more rows (18000), which is why its held-out MAEs are lower; they are not comparable to the paper's nTrain = 2500. Under the exact protocol the CAREFL numbers are stable across init seeds (spread ≈ 0.03) and the x4 query is 1.5–2× worse than under the per-node approximation — the reference's rule, faithfully applied, is not the best optimizer for this DGP, and the paper's Fig. 6 is a visual match either way. With D1 switched off (Bernstein domain on train min/max, `RANGE_Q = 0`, under the per-node protocol): x3 MAE 0.083 / 0.038 / 0.050 (better), x4 0.143 / 0.083 / 0.154 (worse), val NLL 1.444 / 1.476 (worse). The quantile domain stays the default; a `range_quantile` knob would make R's choice available.

Runtime and the epochs

Triangle: 500 epochs × 1250 steps of batch 32 — 3.2 s per epoch single-process at 2–4 torch threads (5.5 s at 32 threads, the step is overhead-bound), 42–69 min per variant on the 2-core CI runners; VACA 4.7 min, CAREFL 6.6 min; the workflow's ten jobs run in parallel, 70 min wall against a 300-min cap.

The one deviation taken for CI runtime is the triangle **batch size and learning rate**: batch 256 at lr 0.004 instead of the paper's batch 32 at lr 0.001, the same 500 epochs in 8× fewer optimizer steps. Measured on 2026-08-26 against the committed ground-truth bands (glorot init, cs max err unless noted):

batch / lr / epochs	linear-ls β13	linear-cs	atan-cs	mixed exp-cs (TV)	steps vs paper
32 / 0.001 / 500 (paper)	−0.170	0.110	0.080	0.071 (0.019)	1×
128 / 0.001 / 500	−0.170	0.170	0.041	0.126 (0.023)	1/4
128 / 0.004 / 500	−0.178	0.160	0.094	0.070 (0.014)	1/4
128 / 0.004 / 250	−0.170	0.147	0.070	0.131 (0.035)	1/8
256 / 0.004 / 500 (CI)	−0.171	0.132	0.073	0.072 (0.017)	1/8
256 / 0.008 / 500	−0.181	0.163	0.113	0.073 (0.017)	1/8
512 / 0.010 / 500	−0.169	0.183	0.104	0.119 (0.015)	1/16

Batch 256 / lr 0.004 is the only row that keeps every cs error within 0.02 of the paper protocol; larger batches or rates bend the misspecified linear-cs curve (0.16–0.18) and lr 0.008 hurts atan-cs. On CI (run 32974872751) the triangle jobs take 7–11 min instead of 42–69, the workflow about 12 min wall instead of 70. The grid ran with glorot init; the committed configs use `init: normal` (the triangle scripts' initializer), whose CI-config numbers are in the result tables above (cs errors 0.088 / 0.077 / 1.024 / 0.107). The paper-protocol numbers stay in this document as the reference; the ground truth is pinned from the CI config.

An epoch cut was measured as well: at 100 epochs the LS coefficients are unchanged (they settle after ~40 epochs, Fig. 14) but the complex shift is not — cs max err 0.235 (linear-cs, 0.116 at 500) and 0.395 (mixed exp-cs, 0.126); at 250 epochs 0.112 and 0.203. The paper's 500 stay.

Repository choices the paper does not state

Seeds; the validation draw sizes for the triangle scripts (the R code's `test`); `n_compare`; `n_heldout = 300` and the α set {−1.5, 0, 1.5} scored next to the single `x_obs`; the mixed cutpoints from the R code; the odds-ratio sample (40000 rows) and the counterfactual-PMF sample (2000 rows × 200 draws).

Per-observation scores & the effect-modifier scan

`flow.scores(df, node)` returns the score $\psi_i = \partial \ell_i / \partial \theta$ of each observation for the node's interpretable shift coefficients. These coefficients are every `LS` weight and every `VC` term's `beta0`. An `LS` weight gives one column per continuous parent and one column per one-hot level of an ordinal parent. The `beta0` column is named after the treatment.

The computation is **analytic and exact**, with no autograd. Shifts enter the latent additively. Thus $\partial \ell_i / \partial \beta = (\partial \ell_i / \partial s_i) \cdot x_i$, with $\partial \ell_i / \partial s$ in closed form (`src/tramdag/scores.py`). The function is a pure read-out. It does not touch the fitting or sampling code paths. At a fitted MLE, the sum of each column is ≈ 0. Tests pin this behavior and include a float64 finite-difference check.

Worked example: where does β(x) need to bend?

The score is the cheapest effect-modifier detector available. It uses the structural-change logic of model-based recursive partitioning (Zeileis & Hornik; Dandl et al. 2024). Before you fit an expensive model, fit the **cheap all-`LS` model**. This fit takes seconds and is deterministic. Then scan the treatment-coefficient scores:

```
flow = td.CausalFlowDAG(all_ls_spec, seed=0)
flow.fit_classical(df)  # seconds, exact MLE

scan = flow.effect_modifier_scan(df, node="Y", t="T")
#         stat   p_value  crit_5pct   flag
# X2      4.21    <0.001     1.3581   True     <- true modifier
# X3      3.05    <0.001     1.3581   True     <- true modifier
# X1      0.74     0.64      1.3581  False     <- prognostic only
```

For each candidate covariate, the scan orders the scores by that covariate and forms the scaled cumulative sum `B_j = Σ_{i≤j} ψ_(i) / (sd(ψ)·√n)`. Under a stable coefficient, `B` is a Brownian bridge. Thus `sup|B|` has the Kolmogorov distribution (5% critical value 1.358). A coefficient that truly varies with the covariate makes the ordered scores drift. The flagged covariates are the measured shortlist of `VC` modifiers (`docs/varying-coefficients.md`):

```
spec["Y"] = td.ContinuousNode(
    td.CS("X1", "X2", "X3") + td.VC("X2", "X3", t="T")
)  # scan-informed
```

Read it as screening, not as a test of effect modification. The statistic measures how unstable the *cheap* model is along a covariate, and instability has two sources: a coefficient that genuinely varies, and a **prognostic part the cheap model gets wrong**. In the example above `X1` is unflagged because its prognostic effect is linear; make it quadratic and `X1` flags as strongly as the true modifiers (0.56 → 4.14 on the DGP of `notebooks/varying_coefficients.py`, which demonstrates both cases side by side). A flag therefore means "look here", and what you find may be a modifier or a misspecification — both worth knowing.

Notes: For a binary ordinal `LS` treatment, `t` resolves to the identified level-1-vs-0 contrast. For a `VC` treatment, `t` resolves to `beta0`. Candidates default to every column of `df` except the node and `t`. Modifiers do not need to be parents. For few-level (heavily tied) candidates, the ordering is only partial. Then read the scan as a ranking diagnostic, not as an exact-size test. You can also use the scores later for influence-function analyses and robust standard errors.

How fast can a CausalFlowDAG train?

This document benchmarks learning-rate schedules, per-node freezing, batch sizes, devices, and LBFGS. The benchmark ran in June 2026 on an Apple-silicon Mac mini with torch 2.12 (CPU unless noted). To reproduce it, run `experiments/benchmarks/bench_training.py` (`cd experiments && uv run python -m benchmarks.bench_training`, or `--quick` for one seed on cpu); the full grid takes ≈ 35 min. For a quick **cross-machine** comparison, use the self-contained `experiments/benchmarks/perf_machine.py`. It runs fixed 200-epoch workloads on all available devices and writes a machine fingerprint to JSON, and it needs nothing but `pip install tramdag`. The raw CSV is a local run artifact and is not committed.

The recipes, and where they live now

The recipes this benchmark compares were `fit()` options in 0.3 (`schedule="plateau"`, `freeze_patience=`, `restore_best=`). In 0.4 `fit` is one minibatch Adam loop and the recipes are **callbacks** — training strategies, not part of the model (see fitting.md):

recipe	behavior	today
constant	one lr for the whole run	`flow.fit(train, epochs=, learning_rate=)`
plateau+freeze	**per-node**: a node whose own validation NLL hasn't improved by `min_delta` (1e-4 default; the stroke run uses 1e-5, see the benchmark) for `patience` epochs gets its lr × 0.3, floored at 1e-3 × the start; once decayed ≥ 100× and flat for `freeze` epochs it leaves training (rate 0). Valid because the per-node losses have independent gradients.	`tramdag.callbacks.PerNodePlateau`, a callback over one parameter group per node (`per_node_adam`), measured in `experiments/benchmarks/bench_training.py`
global plateau	torch's `ReduceLROnPlateau` on the summed validation NLL — the paper reference's rule	`experiments/paper/helpers.py::fit_paper`

`"onecycle"` and `"cosine"` were part of this benchmark and lost to `"plateau"` on every workload; 0.4.0 removed both. The result tables below keep their measured rows as the record of that decision.

The exact-MLE path: for an exact comparison with classical methods (`statsmodels`, R `polr`/`tram`) no recipe is needed —

```
flow.fit(train_df, epochs=4000, learning_rate=1e-2, batch_size=512)  # constant lr
flow.fit(train_df, epochs=2000, learning_rate=1e-3, batch_size=512)  # 2nd phase
```

is still the exact-MLE path (`experiments/misc/validate_ls.py` runs a three-phase variant of it, 4000/2000/1000 epochs at 1e-2/1e-3/1e-4, batch 256). `fit` keeps the final weights. The guard test `tests/test_fit_hooks.py::test_torch_plateau_scheduler_preserves_exact_mle` shows that a schedule through the hooks still lands the all-`ls` fit on the classical MLE within the usual tolerances.

LBFGS is *not* a `fit()` option — it ships as its own method. `fit_classical` runs float64 full-batch L-BFGS with a strong-Wolfe line search and supersedes the hand-rolled recipe this report benchmarked: it is deterministic, lands on the exact MLE, matches `statsmodels`/R `polr`, and refuses non-`ls` specs (minibatch noise also regularizes the MLPs, so flexible models belong in `fit`). The float32 single-precision variant measured here was fast (< 2 s) but not robust across seeds (Findings #2) — `fit_classical`'s float64 upcast is what fixed that.

```
report = flow.fit_classical(train_df)  # exact classical MLE, seconds
```

Method: time-to-target, not loss-go-down

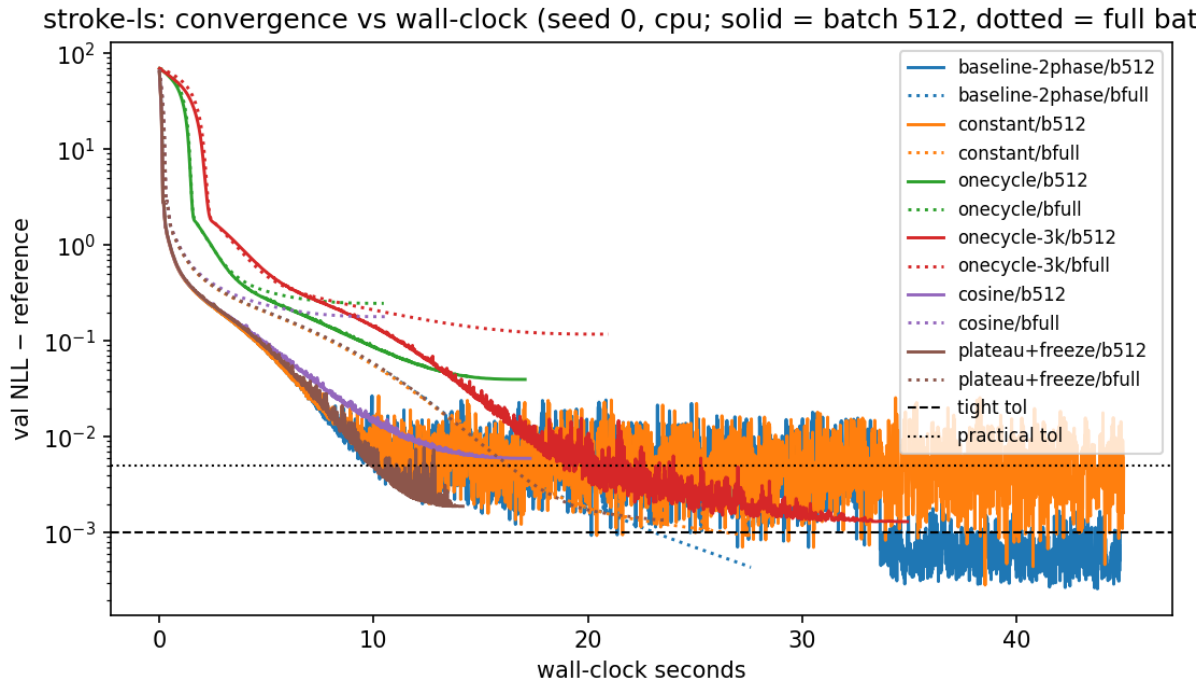
Each config runs once. The benchmark's callback records per-epoch validation NLL *and* wall-clock time. From these records, we measure the seconds until the fit is within a fixed gap of a cached long-run reference (3 torch seeds, medians):

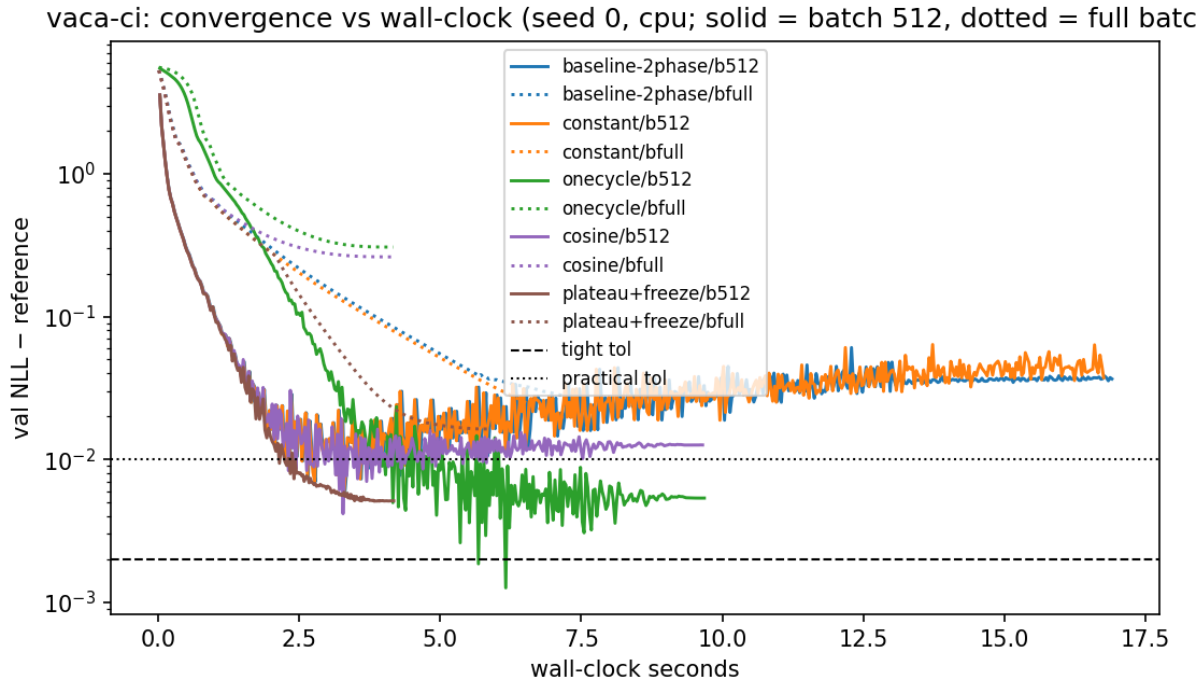
workload	model / data	reference NLL	tight tol	practical tol
stroke-ls	all-`ls` 5-node DAG, frozen `experiments/misc/data/magic-mrclean/ls` (n=1275, full-data MLE)	10.3042 (train)	+1e-3	+5e-3
vaca-ci	all-`ci` flow, frozen `experiments/paper/data/vaca` (n=5000, 90/10 split)	4.9632 (val)	+2e-3	+1e-2

Tight ≈ exact-MLE equivalence (statsmodels/R-polr match). *Practical* ≈ coefficient-equivalent: a fit with gap ≈ 3e-3 already matches the R reference coefficients within the tolerances of `experiments/misc/validate_ls.py`. The same exact-MLE-under-plateau-and-freezing property is pinned on an inline DGP by `tests/test_fit_hooks.py::test_torch_plateau_scheduler_preserves_exact_mle`.

These numbers were measured before the experiment code moved into `experiments/benchmarks/`; the workloads are unchanged (same frozen data, same specs), so the timings still stand. Re-running the benchmark reproduces the machine-independent part exactly: the stroke-ls reference NLL 10.3042.

Results





The table shows median seconds to target (batch 512, cpu). A "—" entry means that the fit never reached the target within the budget.

config	stroke-ls practical	stroke-ls tight	vaca-ci practical	vaca-ci tight	self-stops
baseline two-phase (old default)	9.0	**21.4**	2.1	2.8[1]	no (runs 40 s / 15 s)

config	stroke-ls practical	stroke-ls tight	vaca-ci practical	vaca-ci tight	self-stops
constant 1e-2	9.1	21.5[3]	2.2	2.8[1]	no
onecycle (1500 / 300 ep)[2]	—	—	3.5	4.5	no
onecycle (3000 ep)[2]	16.8	— (gap 1–2e-3)			no
cosine[2]	—	—	2.2	3.5[1]	no
plateau + freeze	**8.9**	— (gap 2e-3)	**2.0**	2.9	**yes — 13 s / 4 s total**
LBFGS (full-batch)	**1.6** (2/3 seeds)	— (gap 4–8e-3)	n/a	n/a	yes

[1] transient: the val-NLL curve dips through the target and then drifts away. Stroke needs the 1e-3 phase to *stay*. Vaca shows mild overfitting. Final gap for the vaca baseline is 0.037. The old 520-epoch budget **underfits** vaca by ~0.03 nats. Plateau+freeze *stays* at its target. [2] `onecycle` and `cosine` were removed from `fit()` in 0.4 — they lost to plateau on every workload here — so these three rows cannot be re-measured. [3] constant lr at batch 512 stalls at gap 3–7e-3. Only the lr-decay phase closes the last decade. This is why the two-phase recipe existed.

Findings

1. **Per-node plateau decay + freezing is the best default-style trainer.** It has the same time-to-accuracy as the hand-tuned two-phase schedule, but it needs **no budget tuning**. It decays the lr of each node off its own validation curve. It freezes converged nodes, which is a real FLOP saving because the per-node NLLs have independent gradients. And it **stops itself**: 13 s total vs 40 s for the baseline on stroke-ls, 4 s vs 15 s on vaca-ci, at equal or better final NLL.
2. **LBFGS is spectacular but not robust.** Full-batch LBFGS reaches coefficient-level accuracy on the classical all-`ls` model in **< 2 s** (vs 9 s for Adam) on 2/3 seeds. The third seed stalls at gap 8e-3. An Adam warm start made it *worse* (different basin) on every seed. Use it as a fast first shot with the plateau trainer as fallback, not as the default.
3. **OneCycle is a "spend exactly this budget" scheduler.** Accuracy arrives only at the end of its anneal. At 1500 epochs it misses everything. At 3000 epochs it lands gap 1–2e-3. But you must know the right budget in advance, and this requirement is the problem that we try to remove.
4. **Full-batch loses on time-to-target** despite ~1.6× higher epoch throughput. It makes too few optimizer steps per second of compute at these n. Batch 512 is a good default. Very large batches (16k) only improved raw throughput at n=50k.
5. **MPS (Apple GPU) is 3–4× slower than the M-series CPU** at these model sizes (verified correct: identical reconstruction). Kernel-launch overhead dominates sub-millisecond ops. Stay on CPU locally. CUDA on Colab-class GPUs is a different regime.
6. **The old defaults waste or under-spend.** Stroke: 4000 epochs budgeted, converged work done after ~1500 (freezing recovers the difference automatically). Vaca: 520 epochs budgeted, ~0.03 nats short of converged. Fixed budgets are wrong in both directions. Adaptive stopping fixes both.

Recommendation

For everyday fits:

```
opt = torch.optim.Adam(flow.parameters(), lr=1e-2)
plateau = torch.optim.lr_scheduler.ReduceLROnPlateau(opt, factor=0.3, patience=30)

def on_epoch(flow, epoch, opt):
    plateau.step(sum(flow.nll(val).values()))
    return opt.param_groups[0]["lr"] < 1e-5

flow.fit(train, epochs=4000, batch_size=512, optimizer=opt, after_epoch_callbacks=on_epoch)
```

The generous `epochs` value is only a ceiling; the callback stops the fit. For exact classical comparisons where the last 1e-3 matters, append a short constant-lr polish phase (`epochs=500, learning_rate=1e-3`). The per-node variant with freezing is the shipped `tramdag.callbacks.PerNodePlateau`.

One finding is a package default: `epochs` has no default at all, because finding 6 is precisely that a fixed budget cannot be right for every workload. The paper scripts follow the paper's own protocol (one constant-lr run, or the reference's ReduceLROnPlateau for VACA/CAREFL); `validate_ls` runs a three-phase constant-lr descent. Each states its recipe in its own YAML.

Varying-coefficient treatment effects: `VC(*modifiers, t=, penalty=)`

A `VC` term gives a node a **treatment-effect head with its own bias–variance budget** (issue #28). The term contributes

```
beta(modifiers) * x_on        with    beta(x) = beta0 + b_theta(x)
```

to the node's shift. Here `b_theta` is a deliberately small (one 16-unit hidden layer), **penalized** network. The point is not expressiveness. The point is that the flow estimates the effect function *with care*, not as a by-product:

```python
import tramdag as td

spec = {
    "X1": td.ContinuousNode(),
    "X2": td.ContinuousNode(),
    "X3": td.ContinuousNode(),
    "T": td.OrdinalNode(2, td.LS("X1") + td.LS("X2")),
    "Y": td.ContinuousNode(
        td.CS("X1", "X2", "X3")  # prognostic part g(x): as flexible as you like
        + td.VC("X2", "X3", t="T", penalty=1.0)  # effect beta(X2, X3) * T
    ),
}
flow = td.CausalFlowDAG(spec, seed=0).fit(
    train, epochs=500, learning_rate=1e-2, batch_size=512
)  # best-validation weights: callbacks.RestoreBest, see fitting.md

beta = flow.varying_coef(df_new, "Y")  # (n,) array beta(x) — deterministic, y-free
beta0 = float(flow.nodes["Y"].shifts["T"].beta0)  # interpretable main effect
```

`X2` and `X3` appear **twice**: as prognostic parents through `CS` and as effect modifiers through `VC`. This pattern is intended. Only the treatment node named by `t=` owns its edge, and a second term that declares it raises an error. Modifiers can repeat.

Why not `CS("T", "X2", "X3")`? (anti-pattern)

For a binary treatment, the multi-parent `CS` is equivalent in *expressiveness*. Any shift decomposes exactly as `s(x,t) = s(x,0) + [s(x,1) − s(x,0)]·t`. But the `CS` form has **no effect-specific regularization**. The likelihood rewards a good average fit of `s(x,t)`, and nothing rewards a smooth difference between the arms. The read-out is thus the difference of two jointly fitted, unregularized networks, which amplifies noise.

On the heterogeneous-effect validation DGP (see below), the `CS` reduced form reaches corr ≈ 0.5 against the true effect function (tramdag-simu#18 / PR #21). This holds **even when the model is exactly in-class**. The `VC` term reaches corr ≈ 0.99 on the same protocol (`tests/test_vc_term.py`, acceptance bar 0.9). Causal forests and R-learners work not because they *target* the effect, but because they **regularize** it (Nie & Wager 2021, Athey–Tibshirani–Wager 2019). `VC` brings that ingredient into the TRAM framework.

Semantics

- **Scale**: `beta(x)` lives on the node's latent (log-odds) scale. The flow adds it for a continuous node (`u = h(y) + …`) and subtracts it from the cutpoints for an ordinal node, exactly like an `LS` weight. With no modifiers, `VC(t="T")` *is* `LS("T")` (identical model, testably bit-exact). Thus `VC` versus `LS` is a nested question.

- **Penalty**: the fitting objective is the penalized likelihood `Σᵢ NLLᵢ + penalty · ‖b_theta weights‖²` on the total-NLL scale. This is a fixed Gaussian prior whose shrinkage vanishes as n grows. The penalty never applies to `beta0`. `penalty → ∞` shrinks `b_theta` to the zero function and recovers the classical `LS` fit. The default is `penalty=1.0`. If the modifiers are many or n is small, raise the penalty.
- **Identification / centering**: a constant moves freely between `beta0` and `b_theta`. The head's output layer is zero-initialized (`beta(x) = beta0` at step 0). After `fit`, the flow re-centers the head to mean zero over the training data, which preserves the function. Thus `beta0` is the main effect in the training population, the `Colr` reading when `beta` is constant.
- **Warm start** (optional, two lines): fit the all-`ls` version of the node classically and copy the treatment weight into `flow.nodes[node].shifts[t].beta0` before `fit`, so training starts at the classical answer and only learns deviations. Measured on the `vc_hetero` DGP: `beta0` lands within 0.15 of the truth with it, 0.16 from the zero start; the recovery correlation is 0.99 either way.
- **Treatments**: the treatment `x_on` can be continuous or binary (2-level) ordinal. The term is linear in `x_on`. A binary ordinal enters as its 0/1 level, so `beta` is the identified level-1-vs-0 contrast. Multi-level ordinal treatments are a planned follow-up.
- **Read-out**: `flow.varying_coef(df, node, t=...)` evaluates `beta0 + b_theta(modifiers)` in closed form. The result is deterministic and y-free, and it needs no abduction. For a binary treatment, it equals the abduct-difference `u(x, t=1, y) − u(x, t=0, y)` identically (pinned by a test).

Propensity-centered VC: `center=True` (R-learner orthogonalization)

```
td.VC("X2", "X3", t="T", penalty=1.0, center=True)
# contributes  beta(x) * (t - e_hat(x))  to the shift; e_hat comes from you
```

This is Robinson/R-learner centering inside the likelihood. Dandl et al. (2024) found treatment-centering to be *the* decisive ingredient for effect estimation under confounding in model-based forests. That finding reproduces here. The test DGP is strongly confounded, and the model deliberately under-specifies its prognostic part (true g(x) quadratic, model linear). On that DGP, the uncentered β̂ absorbs the confounded misfit. The mean |β̂ − τ| is ≈ 1.1–1.2, which effectively destroys the effect estimate. The centered β̂ stays near truth (≈ 0.1–0.3), a **5–10× bias reduction** (`tests/test_vc_centered.py`). If the prognostic part is correctly specified, centering changes little. Centering is insurance against the misspecification that you do not know you have.

The naive implementations are wrong. Thus this is a **two-stage frozen** design:

- **Training** uses **out-of-fold** ê that *you* compute and pass as `fit(vc_ehat={"Y": {"T": e_oof}})`, one value per training row: any propensity model, each fold predicted by a fit that never saw it. This is the DML cross-fitting requirement — in-sample ê reintroduces the own-observation bias and can be *worse* than no centering. Six lines with the flow's own classical fit:

  ```
  fold_id = np.random.default_rng(0).permutation(len(train)) % 5
  e_oof = np.empty(len(train))
  for j in range(5):
      proxy = td.CausalFlowDAG(t_spec, seed=0)          # the treatment node's spec
      proxy.fit_classical(train.iloc[fold_id != j][["X", "T"]])
      e_oof[fold_id == j] = proxy.pmf(train.iloc[fold_id == j], "T")[:, 1]
  ```

  The OOF values enter the outcome loss as frozen data. Thus **no gradient reaches the treatment node** from the outcome node, and the per-node factorization stays intact (pinned by a gradient-isolation test). `fit` refuses a centered spec without `vc_ehat` and a `vc_ehat` that does not match the spec.

- **Inference** (`log_prob` / `sample` / `abduct` / `pmf` / `scores`) recomputes ê from the flow's **own fitted treatment node**, the full-data fit (the standard DML train/predict split). The computation is detached and uses the current parent values. Under `do(T=t)`, the flow re-derives the regressor as `t − ê(x)`. The flow caches nothing.
- **Interpretation**: with centering, `beta0` is the effect at the treatment margin (the observed propensities). `varying_coef` is unchanged, because centering moves the regressor, not β. The LS-nesting reading applies to the uncentered term only. Centering requires a binary ordinal treatment. Continuous-treatment centering with E[T|x] is a follow-up. `center=False` (the default) is bit-identical to the uncentered term.

Validation

The `vc_hetero` DGP in `tests/conftest.py` is a logistic-shift SCM with known `beta_true(x) = −1 + 0.8·X2 − 0.6·X3`. It has a nonlinear prognostic part and confounded assignment (X2 is confounder *and* modifier), which is the configuration where the `CS` reduced form fails hardest. Acceptance (`tests/test_vc_term.py`) requires three results: recovery corr ≥ 0.9 at n = 5000 (measured ≈ 0.99, min over 3 seeds 0.986), a fitted `beta0` that matches `fit_classical` under a large penalty, and the read-out identities. The centering claims are measured against the `confounded` DGP in the same file. Both former follow-ups have since shipped: propensity centering (#30) is `center=True`, documented above, and the per-observation scores for effect-modifier scans (#29) are `flow.scores` and `flow.effect_modifier_scan` — see scores.md.

Upstream candidates for zuko

What tramdag works around in zuko (1.6.0), ranked by value/effort as candidate upstream PRs/issues. tramdag uses zuko surgically — only `zuko.transforms` (`BernsteinTransform`, `MonotonicRQSTransform`, `MonotonicAffineTransform`) behind the three wrappers in `src/tramdag/transforms.py`; the flow machinery, base distribution and DAG structure are tramdag's own. Suggested order of attack: 4 → 1 → 3 → 2 → 5.

1. Analytic `call_and_ladj` for `BernsteinTransform`

`BernsteinTransform` inherits `MonotonicTransform.call_and_ladj`, which gets the Jacobian by `torch.autograd.grad` in the forward pass. That roughly doubles the training graph and makes the per-node loss un-`torch.compile`-able (double backward; see docs/research/REPORT.md). The derivative is closed form (`f'(x) = order·Δθ` against the order−1 basis — zuko already computes it in `_offset_and_slope` for the tail slopes), so an override is ~25 lines with no API change, consistent with `MonotonicRQSTransform` which already has one. tramdag gets a free speedup and the `torch.compile` axis back.

2. Learnable/linear tails for `MonotonicRQSTransform`

zuko pads the knot derivatives with `exp(0)=1` and extrapolates as the identity outside `[-B, B]`, so the tail slope is fixed at 1 regardless of θ — the structural reason `spline` consistently trails `bernstein` here (~10% of data sits beyond the 5%/95% pre-scaling range; CLAUDE.md, `spec.py`, demo notebook section 6). Upstream: accept boundary derivatives (shape `(*, K+1)`) or an opt-in `tails="linear"`; identity tails are deliberate in the NSF design, so the framing must be back-compatible. Would delete the caveats and make `spline` a first-class basis choice.

3. Public inverse of `_constrain_theta`

`BernsteinUT.marginal_init_theta` closed-form-inverts zuko's *private* `_constrain_theta` (cumsum-of-softplus), hard-coding its internal `log(2)·n/2` centering — any upstream reparametrization silently breaks the calibrated start. A public `unconstrain_theta(theta)` (or a `linear_theta(a, b, n, bound)` classmethod) is ~15 lines upstream and removes the private-detail coupling. Framing: "identity/linear initialization support", a standard flow trick.

4. Docstring fix: the Bernstein θ-shape off-by-one

zuko's docstring says θ has shape `(*, M-2)` for a degree-M polynomial; `n` unconstrained coefficients become `n+2` control points = degree `n+1`, so the correct claim is `(*, M-1)`. This off-by-one is a real replication trap (order 21 vs the paper's `len_theta=20` = order 19) that costs tramdag prose in three docs. Trivial PR, near-certain accept.

5. `Logistic` in `zuko.distributions`

Neither zuko nor torch ships a Logistic distribution; tramdag hand-rolls `StandardLogistic` (~50 lines: `log_prob`, `sample`, `icdf`). Logistic latents are standard in transformation models and discrete flows, and zuko has precedent (`GeneralizedNormal`) — but zuko may point at `TransformedUniform(SigmoidTransform().inv)`, and tramdag would keep its `_U_EPS` clamp locally either way. Lowest value of the five.

Anti-candidates (look upstreamable, are not)

- **Quantile pre-scaling / `_ScaledUT`**: a modeling choice; zuko's `ComposedTransform` with `MonotonicAffineTransform` already composes it.
- **The ordinal ordered-logit transform and log-space `ordinal_log_prob`**: not a bijection, outside zuko's flow scope (torch.distributions material, if anywhere).
- **LS/CS/CI/VC terms, marginal-init policy, propensity centering, scores**: TRAM-DAG semantics. zuko's `MaskedMLP` already supports arbitrary adjacency; tramdag doesn't use it because it needs per-term interpretability.
- **Spline `slope` / Bernstein `eps` knobs**: already exposed upstream.